CS 6160 Cryptology: Modern Cryptology

Shibam Ghosh

October 30, 2026

Department of CSE, IIT Hyderabad



Introduction

What is Cryptology?

- **Cryptography:** The art of writing in secret characters.
  - Greek word “kruptos” meaning hidden, and “graphia”, meaning writing.
- **Cryptanalysis:** The analysis and deciphering of secret writings.
- **Cryptology:** The scientific study of both of the above.



In our context, cryptology is the scientific study of the **protectionofinformation**.

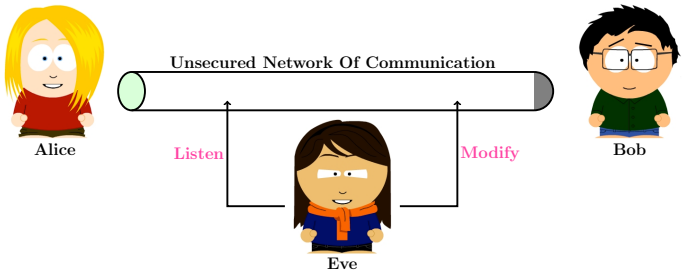
What is Cryptology?

- **Cryptography:** The art of writing in secret characters.
  - Greek word “kruptos” meaning hidden, and “graphia”, meaning writing.
- **Cryptanalysis:** The analysis and deciphering of secret writings.
- **Cryptology:** The scientific study of both of the above.



In our context, **cryptology** is the scientific study of the **protectionofinformation**.

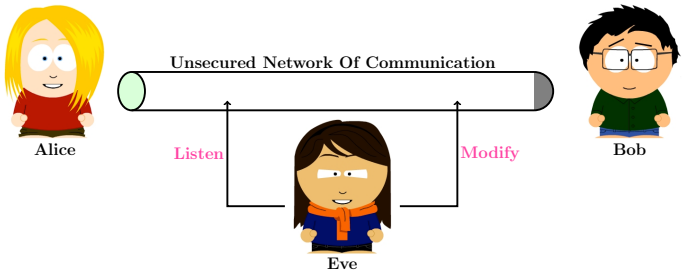
Why Cryptography?



The Goal is to develop **Efficient** algorithms with the following functionalities:

- **Confidentiality:** Prevents unauthorized disclosure of information.
- **Integrity:** Ensuring data has not been modified without detection.
- **Authenticity:** Ensures that the recipient can verify the identity of the sender.

Why Cryptography?



The Goal is to develop **Efficient** algorithms with the following functionalities:

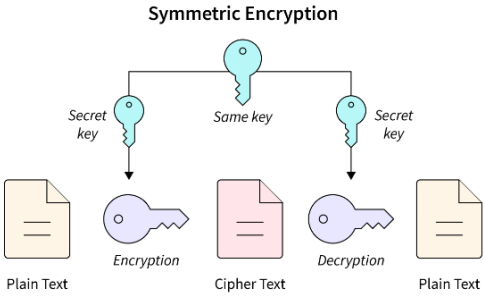
- **Confidentiality:** Prevents unauthorized disclosure of information.
- **Integrity:** Ensuring data has not been modified without detection.
- **Authenticity:** Ensures that the recipient can verify the identity of the sender.

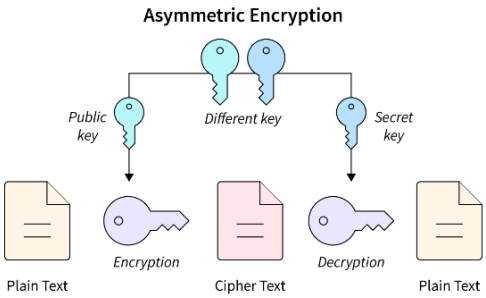
Applications of Cryptography

1. **Secure communication**
   (encryption, authentication, messaging, email)
2. **Data security**
   (files, databases, cloud storage)
3. **Authentication and access control**
   (passwords, PINs, MFA)
4. **Digital identity**
   (secure identification, certificates)
5. **Electronic commerce**
   (online shopping, secure payments)
6. **Banking and digital payments**
   (electronic funds transfer, digital cash)
7. **Blockchain and cryptocurrencies**
8. **Electronic voting**
9. **Digital signatures and contract signing**
10. **Copyright and digital rights management**
11. **Secret sharing**
12. **Zero-knowledge proofs**
13. **Secure multi-party protocols**
14. Many more emerging applications

Modern Cryptography

Symmetric and Asymmetric Key Cryptography





Modern Crypto Infrastructure

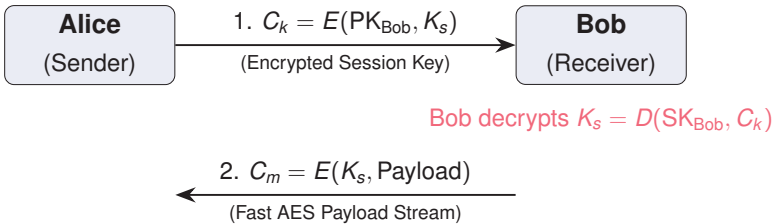
The Core Principle

Symmetric-key cryptography is ‘fast’, but need a pre-shared secret key. Use **Asymmetric** cryptography to exchange a short-lived secret key, then use **Symmetric Encryption** to encrypt the actual payload data.

Modern Crypto Infrastructure

The Core Principle

Symmetric-key cryptography is 'fast', but need a pre-shared secret key. Use **Asymmetric** cryptography to exchange a short-lived secret key, then use **Symmetric Encryption** to encrypt the actual payload data.



Real-World Usage

- **HTTPS / TLS Handshake:** The foundation of secure web browsing.
- **SSH (Secure Shell):** Uses RSA/ECDSA to negotiate AES keys for terminal access.
- **PGP / GPG Email Security:** Encrypts large email attachments with symmetric keys while wrapping the key asymmetrically.

Key Takeaways

- Asymmetric crypto provides **key establishment**
- Symmetric crypto provides **efficient data transfer**



Symmetric Key Cryptosystem

Formal Definition: Symmetric Key Cryptosystem (Cipher)

A symmetric key cryptosystem is five-tuple:

$$(\mathcal{P}, \mathcal{C}, \mathcal{K}, \mathcal{E}, \mathcal{D})$$

- $\mathcal{P}$ is a finite set of possible **plaintexts**.
- $\mathcal{C}$ is a finite set of possible **ciphertexts**.
- $\mathcal{K}$ is a finite set of possible **keys**.
- $\mathcal{E}$ and $\mathcal{D}$ are set of deterministic algorithms

For each $K \in \mathcal{K}$, $\exists$ an encryption rule $E_K \in \mathcal{E}$ and a decryption rule $D_K \in \mathcal{D}$:

$$E_K : \mathcal{P} \to \mathcal{C} \quad \text{and} \quad D_K : \mathcal{C} \to \mathcal{P} \text{ such that } D_k(E_k(P)) = P \quad \forall P \in \mathcal{P}$$

Formal Definition: Symmetric Key Cryptosystem (Cipher)

A symmetric key cryptosystem is five-tuple:

$$(\mathcal{P}, \mathcal{C}, \mathcal{K}, \mathcal{E}, \mathcal{D})$$

- $\mathcal{P}$ is a finite set of possible **plaintexts**.
- $\mathcal{C}$ is a finite set of possible **ciphertexts**.
- $\mathcal{K}$ is a finite set of possible **keys**.
- $\mathcal{E}$ and $\mathcal{D}$ are set of deterministic algorithms

For each $K \in \mathcal{K}$, $\exists$ an encryption rule $E_K \in \mathcal{E}$ and a decryption rule $D_K \in \mathcal{D}$:

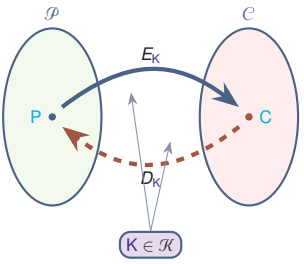
$$E_K : \mathcal{P} \to \mathcal{C} \quad \text{and} \quad D_K : \mathcal{C} \to \mathcal{P} \text{ such that } D_k(E_k(P)) = P \quad \forall P \in \mathcal{P}$$

Formal Definition: Symmetric Key Cryptosystem (Cipher)

A symmetric key cryptosystem is five-tuple:

$$(\mathcal{P}, \mathcal{C}, \mathcal{K}, \mathcal{E}, \mathcal{D})$$

- $\mathcal{P}$ is a finite set of possible **plaintexts**.
- $\mathcal{C}$ is a finite set of possible **ciphertexts**.
- $\mathcal{K}$ is a finite set of possible **keys**.
- $\mathcal{E}$ and $\mathcal{D}$ are set of deterministic algorithms



For each $K \in \mathcal{K}$, $\exists$ an encryption rule $E_K \in \mathcal{E}$ and a decryption rule $D_K \in \mathcal{D}$:

$$E_K : \mathcal{P} \to \mathcal{C} \quad \text{and} \quad D_K : \mathcal{C} \to \mathcal{P} \text{ such that } D_k(E_k(P)) = P \quad \forall P \in \mathcal{P}$$

Kerckhoffs' Principle

Kerckhoffs' Note

Both algorithm sets $\mathcal{E}$ and $\mathcal{D}$ are publicly known

- Assume the attacker knows every structural feature and limit of the cipher
- We do not rely on keeping the design of our algorithm a secret
- Must remain secure even if the adversary steals the source code

The Only Secret is the **Key** denoted as K

Kerckhoffs' Principle

Kerckhoffs' Note

Both algorithm sets $\mathcal{E}$ and $\mathcal{D}$ are publicly known

- Assume the attacker knows every structural feature and limit of the cipher
- We do not rely on keeping the design of our algorithm a secret
- Must remain secure even if the adversary steals the source code

The Only Secret is the **Key** denoted as K



Kerckhoffs' Principle

Kerckhoffs' Note

Both algorithm sets $\mathcal{E}$ and $\mathcal{D}$ are publicly known

- Assume the attacker knows every structural feature and limit of the cipher
- We do not rely on keeping the design of our algorithm a secret
- Must remain secure even if the adversary steals the source code

The Only Secret is the **Key** denoted as K

Cipher Design Criteria

1. Must be **efficient** to encrypt and decrypt whenever the correct key is provided
2. Must be computationally **infeasible** to:
   - ▷ Decrypt data without knowing the key
   - ▷ Extract the key
   - ▷ Extrapolate any statistical information on the plaintext

One-Way Functions

These functions are public algorithms that are computationally **efficient** to compute forward, but exceptionally **difficult** to invert (without the key).

Cipher Design Criteria

1. Must be **efficient** to encrypt and decrypt whenever the correct key is provided
2. Must be computationally **infeasible** to:
   - ▷ Decrypt data without knowing the key
   - ▷ Extract the key
   - ▷ Extrapolate any statistical information on the plaintext

One-Way Functions

These functions are public algorithms that are computationally **efficient** to compute forward, but exceptionally **difficult** to invert (without the key).

Attack Models

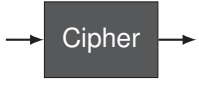
Attack Models

What is an Attack Model?

An **attack model** specifies the capabilities of an attacker:

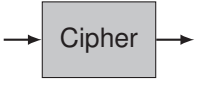
- What information is available?
- What operations can the attacker perform?
- What limitations does the attacker have?

Black-Box Model



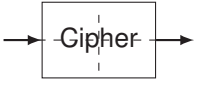
Only inputs/outputs

Gray-Box Model



Partial knowledge

White-Box Model



Complete knowledge

Cryptographic (Black-Box) Attack Models

Passive & Basic Models:

- **Ciphertext-Only (COA):** Attacker observes only ciphertexts C
- **Known Plaintext (KPA):** Access to random, non-chosen plaintext-ciphertext pairs (P, C)

Active Control Models:

- **Chosen Plaintext (CPA):** Attacker encrypts any chosen P get corresponding C
- **Chosen Ciphertext (CCA):** Attacker has access to a decryption oracle to decrypt any chosen C

Advanced Interactive Models:

- **Adaptive Variants (ACPA / ACCA):** Choosing subsequent plaintexts or ciphertexts dynamically *after* observing outcomes of previous queries
- **Related-KeyAttack(RKA):** Attacker queries multiple oracles initialized with different keys sharing a known or chosen mathematical relationship

Cryptographic (Black-Box) Attack Models

Passive & Basic Models:

- **Ciphertext-Only (COA):** Attacker observes only ciphertexts C
- **Known Plaintext (KPA):** Access to random, non-chosen plaintext-ciphertext pairs (P, C)

Active Control Models:

- **Chosen Plaintext (CPA):** Attacker encrypts any chosen P get corresponding C
- **Chosen Ciphertext (CCA):** Attacker has access to a decryption oracle to decrypt any chosen C

Advanced Interactive Models:

- **Adaptive Variants (ACPA / ACCA):** Choosing subsequent plaintexts or ciphertexts dynamically *after* observing outcomes of previous queries
- **Related-KeyAttack(RKA):** Attacker queries multiple oracles initialized with different keys sharing a known or chosen mathematical relationship



Cryptographic (Black-Box) Attack Models

Passive & Basic Models:

- **Ciphertext-Only (COA):** Attacker observes only ciphertexts C
- **Known Plaintext (KPA):** Access to random, non-chosen plaintext-ciphertext pairs (P, C)

Active Control Models:

- **Chosen Plaintext (CPA):** Attacker encrypts any chosen P get corresponding C
- **Chosen Ciphertext (CCA):** Attacker has access to a decryption oracle to decrypt any chosen C

Advanced Interactive Models:

- **Adaptive Variants (ACPA / ACCA):** Choosing subsequent plaintexts or ciphertexts dynamically *after* observing outcomes of previous queries
- **Related-KeyAttack(RKA):** Attacker queries multiple oracles initialized with different keys sharing a known or chosen mathematical relationship

Hill Cipher: Security Analysis

Ciphertext-Only Attacks

- The key space can be made immensely large by choosing matrix elements from a vast set of integers
- What is the size of the key space for $n \times n$ Hill cipher over $\mathbb{Z}_{26}$?

Insecure in the Known-Plaintext Attack Model

Completely vulnerable when plaintext–ciphertext pairs are known. The secret key matrix K can be easily computed using Gaussian elimination.

Security Goals

Security Goal: Indistinguishability (IND)

Informally: A cipher is secure if, even given a large number of plaintext–ciphertext pairs, nothing can be learned about the cipher's behavior

Indistinguishability Game:

Oracle $\mathcal{O}$

1. Choose $b \xleftarrow{\$} \{0,1\}$
2. Receive (P_i^0, P_i^1)
3. Compute $C_i = \mathcal{E}_K(P_i^b)$
4. Return C_i

$\mathcal{O}$

$(P_1^0, P_1^1), (P_2^0, P_2^1), ..., (P_d^0, P_d^1)$ $\quad C_1, C_2, ..., C_d$



After receiving all ciphertexts, the adversary's task is to correctly guess b

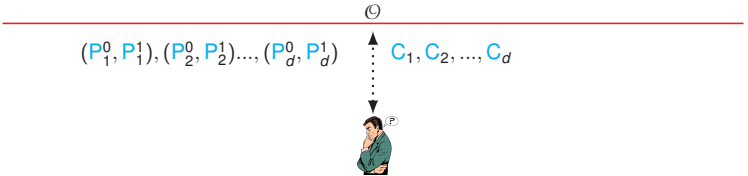
Security Goal: Indistinguishability (IND)

Informally: A cipher is secure if, even given a large number of plaintext–ciphertext pairs, nothing can be learned about the cipher's behavior

Indistinguishability Game:

Oracle $\mathcal{O}$

1. Choose $b \xleftarrow{\$} \{0,1\}$
2. Receive (P_i^0, P_i^1)
3. Compute $C_i = \mathcal{E}_K(P_i^b)$
4. Return C_i

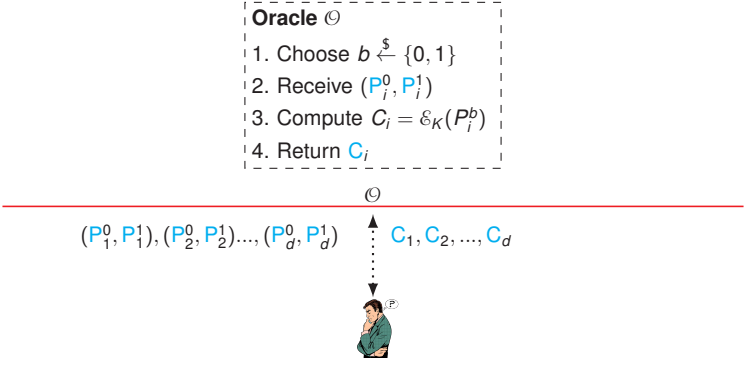


After receiving all ciphertexts, the adversary's task is to correctly guess b

Security Goal: Indistinguishability (IND)

Informally: A cipher is secure if, even given a large number of plaintext–ciphertext pairs, nothing can be learned about the cipher's behavior

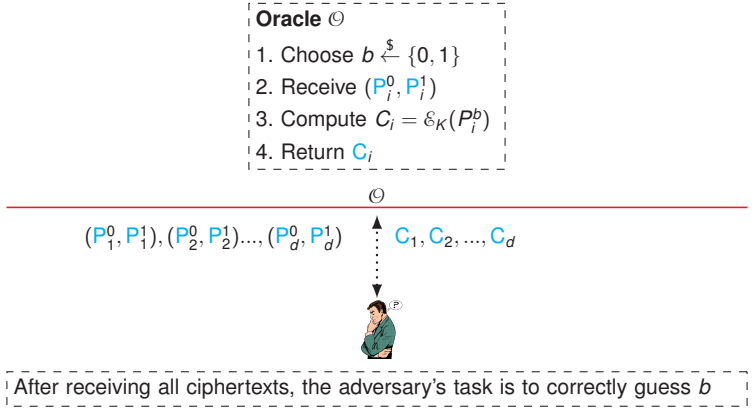
Indistinguishability Game:



Security Goal: Indistinguishability (IND)

Informally: A cipher is secure if, even given a large number of plaintext–ciphertext pairs, nothing can be learned about the cipher's behavior

Indistinguishability Game:



Security Goal: Non-Malleability

An encryption scheme is *non-malleable* if, given a ciphertext $C_1 = E_K(P_1)$, it is computationally infeasible to generate another valid ciphertext $C_2 = E_K(P_2)$, such that the new plaintext P_2 is *meaningfully related* to P_1.

$$P_2 = P_1 \oplus X, \quad \{P_1 = \text{``Transfer Rs. 1000''}, P_2 = \text{``Transfer Rs. 10000''}\}$$

Example

For the one-time pad, $C_1 = P_1 \oplus K$, an attacker can compute $C_2 = C_1 \oplus X$. After decryption,

$$P_2 = C_2 \oplus K = (P_1 \oplus K \oplus X) \oplus K = P_1 \oplus X.$$

Thus, the attacker modifies the plaintext without ever learning it.

Security Goal: Non-Malleability

An encryption scheme is *non-malleable* if, given a ciphertext $C_1 = E_K(P_1)$, it is computationally infeasible to generate another valid ciphertext $C_2 = E_K(P_2)$, such that the new plaintext P_2 is *meaningfully related* to P_1.

$$P_2 = P_1 \oplus X, \quad \{P_1 = \text{``Transfer Rs. 1000''}, P_2 = \text{``Transfer Rs. 10000''}\}$$

Example

For the one-time pad, $C_1 = P_1 \oplus K$, an attacker can compute $C_2 = C_1 \oplus X$. After decryption,

$$P_2 = C_2 \oplus K = (P_1 \oplus K \oplus X) \oplus K = P_1 \oplus X.$$

Thus, the attacker modifies the plaintext without ever learning it.

Attack Complexity Profiles

1. Time Complexity

The total number of basic operations (e.g., encryption runs) required by an attacker.

Note: Measured in **offline mode**, when not actively interacting with the target cryptosystem.

1. Data Complexity

The total volume of information required to initiate and validate the attack.

Note: Typically measured as the **number of oracle queries** (plaintext or ciphertext samples) an adversary must collect.

3. Memory Complexity

The maximum storage capacity required by the attacker's system during computation.

Note: Tracks **ram/disk limits** needed to hold lookup arrays, precomputed states, or large key tables.

An attack only becomes practical if **all three complexity metrics** fall within the adversary's feasible physical resource constraints.

Some Generic Attacks

Brute Force Attack (Exhaustive Search)

A generic key recovery attack that tests the entire key space $\mathcal{K}$.

Setup

Let $\mathcal{P} = \mathcal{C} = \{0,1\}^n$ and $\mathcal{K} = \{0,1\}^\kappa$
$E_K : \{0,1\}^n \to \{0,1\}^n$

Attacker Strategy:

1. Collect a known pair (P, C).
2. For every guess $K_g \in \mathcal{K}$, $C_g = E_{K_g}(\mathbf{P})$.
3. If $C_g = C$, the key is found, Return K_g

Time and Memory Complexity:

- **Time:** 2^κ encryption steps
- **Memory:** Negligible ($\mathcal{O}(1)$)

Data Complexity Requirements:

- Correct key pass with prob. 1
- Wrong key pass with prob. $1/2^n$
- If $\kappa < n$: Data = **1 pair**
- If $\kappa \geq n$: Data $\approx \lceil \frac{\kappa}{n} \rceil$ pairs.

Brute Force Attack (Exhaustive Search)

A generic key recovery attack that tests the entire key space $\mathcal{K}$.

Setup

Let $\mathcal{P} = \mathcal{C} = \{0,1\}^n$ and $\mathcal{K} = \{0,1\}^\kappa$
$E_K : \{0,1\}^n \to \{0,1\}^n$

Attacker Strategy:

1. Collect a known pair (P, C).
2. For every guess $K_g \in \mathcal{K}$, $C_g = E_{K_g}(\mathbf{P})$.
3. If $C_g = C$, the key is found, Return K_g

Time and Memory Complexity:

- **Time:** 2^κ encryption steps
- **Memory:** Negligible ($\mathcal{O}(1)$)

Data Complexity Requirements:

- Correct key pass with prob. 1
- Wrong key pass with prob. $1/2^n$
- If $\kappa < n$: Data = **1 pair**
- If $\kappa \geq n$: Data $\approx \lceil \frac{\kappa}{n} \rceil$ pairs.

Brute Force Attack (Exhaustive Search)

A generic key recovery attack that tests the entire key space $\mathcal{K}$.

Setup

Let $\mathcal{P} = \mathcal{C} = \{0,1\}^n$ and $\mathcal{K} = \{0,1\}^\kappa$
$E_K : \{0,1\}^n \to \{0,1\}^n$

Attacker Strategy:

1. Collect a known pair (P, C).
2. For every guess $K_g \in \mathcal{K}$, $C_g = E_{K_g}(\mathbf{P})$.
3. If $C_g = C$, the key is found, Return K_g

Time and Memory Complexity:

- **Time:** 2^κ encryption steps
- **Memory:** Negligible ($\mathcal{O}(1)$)

Data Complexity Requirements:

- Correct key pass with prob. 1
- Wrong key pass with prob. $1/2^n$
- If $\kappa < n$: Data = **1 pair**
- If $\kappa \geq n$: Data $\approx \lceil \frac{\kappa}{\mathbf{n}} \rceil$ pairs.

Dictionary Attack

Variant A: Key Recovery

1. Fix a chosen plaintext P.
2. **Offline:** Compute and store $C = E_K(\mathbf{P})$ for every possible key $K \in \mathcal{K}$.
3. **Online:** Capture a target ciphertext C corresponding to P, look it up in the table

Variant B: Message Recovery

1. Queries the encryption oracle to map **every possible plaintext** to its respective ciphertext under the secret key.
2. When any encrypted message is intercepted, the attacker checks the dictionary to read the plaintext directly without learning the key.

Trade-Off

Advantage: Negligible online execution time

Disadvantage: Massive, memory complexity.

Time-Memory Trade-Off (TMTO) Attack by Hellman, 1980



Hellman's TMTO Concept

- Combines both strategies to achieve a balance between time and memory while keeping a reasonable success probability
- Instead of an exhaustive dictionary, the attacker precomputes and stores only a **subset** of possible values
- Decreased memory costs additional computations during the online phase
- The exact balance depends on the specific implementation and parameters

TMTO: Setup

Setup

- Let $E : \{0,1\}^\kappa \times \{0,1\}^n \to \{0,1\}^n$ be an encryption algorithm
- Let plaintext size and key size are the same, i.e., $n = \kappa$
  - Just an assumption, also works for $n \neq \kappa$
- For some fixed P, define $f : \{0,1\}^n \to \{0,1\}^n : f(\mathsf{K}) = E(\mathsf{K}, \mathsf{P})$
- The attacker's aim is to invert the function f given $\mathsf{C} = E(\mathsf{K}, \mathsf{P})$
- This is chosen plaintext attack

TMTO: Precomputation Phase

- The attacker guesses m keys $k^1, k^2, ..., k^m$. For each key k^i:
  - The attacker computes $k_2^i = f(k^i) = E(k^i, \mathrm{P})$
  - For each $j = 3, ..., t$, the attacker computes $k_j^i = f(k_{j-1}^i) = E(k_{j-1}^i, \mathrm{P})$

$$
\begin{array}{lccccccccc}
Chain-1: & k^1 = k_1^1 & \xrightarrow{f} & k_2^1 & \xrightarrow{f} & k_3^1 & \xrightarrow{f} & \dots & \xrightarrow{f} & k_t^1 \\
Chain-2: & k^2 = k_1^2 & \xrightarrow{f} & k_2^2 & \xrightarrow{f} & k_3^2 & \xrightarrow{f} & \dots & \xrightarrow{f} & k_t^2 \\
 & \vdots & & \vdots & & \vdots & & & & \vdots \\
Chain-m: & k^m = k_1^m & \xrightarrow{f} & k_2^m & \xrightarrow{f} & k_3^m & \xrightarrow{f} & \dots & \xrightarrow{f} & k_t^m
\end{array}
$$

- Each intermediate point is $k_j^i = f(k_{j-1}^i)$, where $k_0^i = k^i$ is the start
- Keep only (k^i, k_t^i) in a table (sorted by end points)

TMTO: Precomputation Phase

- The attacker guesses m keys $k^1, k^2, ..., k^m$. For each key k^i:
  - The attacker computes $k_2^i = f(k^i) = E(k^i, \mathrm{P})$
  - For each $j = 3, ..., t$, the attacker computes $k_j^i = f(k_{j-1}^i) = E(k_{j-1}^i, \mathrm{P})$

$$
\begin{array}{lccccccccc}
Chain-1: & k^1 = k_1^1 & \xrightarrow{f} & k_2^1 & \xrightarrow{f} & k_3^1 & \xrightarrow{f} & \dots & \xrightarrow{f} & k_t^1 \\
Chain-2: & k^2 = k_1^2 & \xrightarrow{f} & k_2^2 & \xrightarrow{f} & k_3^2 & \xrightarrow{f} & \dots & \xrightarrow{f} & k_t^2 \\
 & \vdots & & \vdots & & \vdots & & & & \vdots \\
Chain-m: & k^m = k_1^m & \xrightarrow{f} & k_2^m & \xrightarrow{f} & k_3^m & \xrightarrow{f} & \dots & \xrightarrow{f} & k_t^m
\end{array}
$$

- Each intermediate point is $k_j^i = f(k_{j-1}^i)$, where $k_0^i = k^i$ is the start
- Keep only (k^i, k_t^i) in a table (sorted by end points)

TMTO: Precomputation Phase

- The attacker guesses m keys $k^1, k^2, ..., k^m$. For each key k^i:
  - The attacker computes $k_2^i = f(k^i) = E(k^i, \mathrm{P})$
  - For each $j = 3, ..., t$, the attacker computes $k_j^i = f(k_{j-1}^i) = E(k_{j-1}^i, \mathrm{P})$

$$
\begin{array}{lccccccccc}
Chain-1: & k^1 = k_1^1 & \xrightarrow{f} & k_2^1 & \xrightarrow{f} & k_3^1 & \xrightarrow{f} & \dots & \xrightarrow{f} & k_t^1 \\
Chain-2: & k^2 = k_1^2 & \xrightarrow{f} & k_2^2 & \xrightarrow{f} & k_3^2 & \xrightarrow{f} & \dots & \xrightarrow{f} & k_t^2 \\
 & \vdots & & \vdots & & \vdots & & & & \vdots \\
Chain-m: & k^m = k_1^m & \xrightarrow{f} & k_2^m & \xrightarrow{f} & k_3^m & \xrightarrow{f} & \dots & \xrightarrow{f} & k_t^m
\end{array}
$$

- Each intermediate point is $k_j^i = f(k_{j-1}^i)$, where $k_0^i = k^i$ is the start
- Keep only (k^i, k_t^i) in a table (sorted by end points)

TMTO Phase 2: Online Phase

Given a ciphertext $C = f(K) = E_K(P)$ under an unknown key K, the attacker iteratively computes:

- **Walk:** Compute the sequence $C, f(C), f^2(C), \ldots, f^{t-1}(C)$. After each step, check if the value matches an **end key** k_t^i in the table for some i.
- **Hit:** Suppose a match is found at step j, such that: $f^{t-j}(z) = k_t^i$. This implies the targeted secret key matches the $(j-1)$-th position of chain i.
- **Reconstruction:** Go to the table, look up the start point k_1^i, step forward $j-1$ times to compute k_{j-1}^i, and test if $E_{k_{j-1}^i}(P) == C$.

TMTO Phase 2: Online Phase

Given a ciphertext $C = f(K) = E_K(P)$ under an unknown key K, the attacker iteratively computes:

- **Walk:** Compute the sequence $C, f(C), f^2(C), \ldots, f^{t-1}(C)$. After each step, check if the value matches an **end key** k_t^i in the table for some i.
- **Hit:** Suppose a match is found at step j, such that: $f^{t-j}(z) = k_t^i$. This implies the targeted secret key matches the $(j-1)$-th position of chain i.
- **Reconstruction:** Go to the table, look up the start point k_1^i, step forward $j-1$ times to compute k_{j-1}^i, and test if $E_{k_{j-1}^i}(P) == C$.

Walk :	–		–	–	–	–	C	$\xrightarrow{f}$	$f(C)$	$\ldots$	$\xrightarrow{f}$	$f^j(C)$
Chain-i :	$k^i = k_1^i$	$\xrightarrow{f}$	$\ldots$	k_{j-1}^i	$\xrightarrow{f}$	k_j^i	$\xrightarrow{f}$	k_{j+1}^i	$\ldots$	$\xrightarrow{f}$	k_t^2
Recon :	$k^i = k_1^i$	$\xrightarrow{f}$	$\ldots$	k_{j-1}^i	– –	– –	– –	– –	– –	– –	– –

Collisions, Chain Merging, and Bounds

A key is *covered* if it belongs to any generated chain. If all chain elements across the table are distinct, the table covers mt keys.

But How Many are Distinct? (Birthday Bound, will be discussed later)

- If $t > 2^{n/2}$, a chain self-collides into a cycle with high probability
- Adding more chains increases the risk of different chains colliding and merging. By the Birthday Paradox, a new chain avoids previous coverage only if:

$$mt \times t = mt^2 \leq 2^n \quad (\textbf{Stopping Condition})$$

- A single table of critical size m covers only a $\frac{mt}{mt^2} = \frac{1}{t}$ fraction of the keyspace.

Hellman's Multi-Table Solution

- Introduce t independent tables using $f_\ell(\mathsf{K}) = L_\ell(f(\mathsf{K}))$
- Memory for storing t tables: $M = mt$
- Time $T = t^2$ as there are t tables and each table has $\approx t$ lookups
- Since $mt^2 \approx 2^\kappa$, the trade-off satisfies the curve: $TM^2 = 2^{2\kappa}$

Success Probability

Accounting for overlaps, a single table realistically covers $0.8mt$ unique keys. Across t tables, success probability (the used key is covered) is bounded by:

$$1 - \left(1 - \frac{0.8}{t}\right)^t \approx 1 - e^{-0.8} \approx \mathbf{0.55}$$

Choosing $t = m = 2^{\kappa/3}$ we have **55**% attack success rate.

Foundational Models of Security

1. Unconditional Security

If it is impossible break the cipher, even given unlimited computation time and memory (e.g., a properly implemented One-Time Pad).

- In most of the modern ciphers, the key space is finite. Thus it is **impossible** to block exhaustive search, hence not **unconditionally secure**.

2. Computational Security

The cipher can be broken mathematically, but the computation requires an impractical amount of time (e.g., millions of years).

- A cryptographic scheme is (t, ϵ)-secure if an attacker performing at most t operations, has a probability of success that is no higher than ϵ, where $0 < \epsilon \leq 1$.

Evolution of Key Bit-Sizes

- Moore's law: Computing efficiency doubles roughly every two years
- **40 Bits (2^{40} keys):** Standard in legacy internet code due to old US export caps. **Completely insecure.**
- **56 Bits (2^{56} keys):** Used by the historic Data Encryption Standard (**DES**). Reliable in 1970, **entirely insecure today.**
- **64 Bits:** Marginal protection; deprecated for new system setups.
- **80 Bits:** Found in old ciphers like the Skipjack.
- **128 Bits (2^{128} keys):** The absolute minimum baseline standard for secure symmetric modern cryptosystems today.
- **AES Standards:** The contemporary Advanced Encryption Standard supports key blocks of **128,192,and256bits**.
- What is the post-quantum standard for secure symmetric cryptosystems?



Thank You for your attention!

Any questions?